Acceso a Ficheros

Acceso a Datos

Índice

- Objetivos
- Introducción
- Gestión de ficheros con APIs de Java
- Gestión de excepciones
- Operaciones básicas con ficheros
- Acceso secuencial
- Acceso Aleatorio y la Clase Properties
- CRUD y MVC
- Bibliografía

Objetivos

- Conocer los distintos tipos de ficheros y sus formas de acceso y gestión.
- Conocer las distintas técnicas y mecanismos de lectura y escritura con ficheros.
- Aprender a usar APIs y clases de Java para manejo de ficheros.
- Introducción en el diseño de programación de MVC.

Introducción

- Los ficheros como sistema de persistencia básico

- TIpos de ficheros según su contenido
  - Texto
    - Codificación binaria
  - Binarios

Gestión de ficheros con API de Java

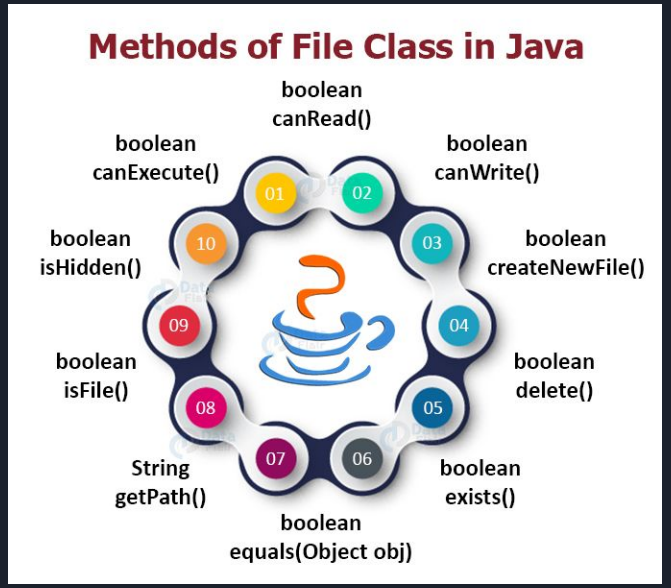
Clase File (java.io.File)

- Constructor Principal: File(String ruta)
    - (hay más opciones)

- Miembros de clase (se dispone de versión en char y en String).
    - pathSeparator / pathSeparatorchar
    - separator/ separatorChar

Gestión de ficheros con API de Java

- Métodos de consulta de propiedades
  - canRead()
  - canWrite()
  - canExecute()
  - exists()
  - isFile()/isDirectory
  - length()
  - getPartent()/getName()/getPath()
- Creación, borrado y renombrado
  - createNewFile()/createTempFile()
  - delete()
  - renameTo()
  - mkdir()



Gestión de ficheros con API de Java

- TAREA PROPUESTA:
  - Hacer un programa en Java que haga la tarea “dir” (o “ls”): listado del directorio.
  - Mejoras:
    - Que reciba por parámetros de qué directorio ejecutar el listado (por defecto, “.”).
    - Que dé más detalles que el nombre del fichero.
    - Que sea recursivo.

Gestión de excepciones

La gestión de excepciones es la estrategia usada por java para gestionar problemas y dar robustez a las aplicaciones.

Las excepciones son eventos que indican errores o situaciones que interrumpen la ejecución del programa. Se avisan de estas para poder manejarlas:

```
Try{

        // Bloque de instrucciones que se quiere monitorizar

}catch( ClaseExcepcion instanciaExcepcion){

        //En caso de suceder la excepción del tiempo ClaseExcepcion, instrucciones a realizar en ese caso.

}catch( ClaseOtraExcepcion instanciaOtraExcepcion){

        //Se pueden tener varios bloques catch, teniendo en cuenta de que se chequean en orden.

}finally{

        //Bloque de instrucciones que se ejecuta al final del try si todo va bien o despues de un catch si pasa una excepcion . Este bloque SIEMPRE se ejecuta, incluso aunque alguna instrucción lance otra Excepcion

}
```

Gestión de excepciones

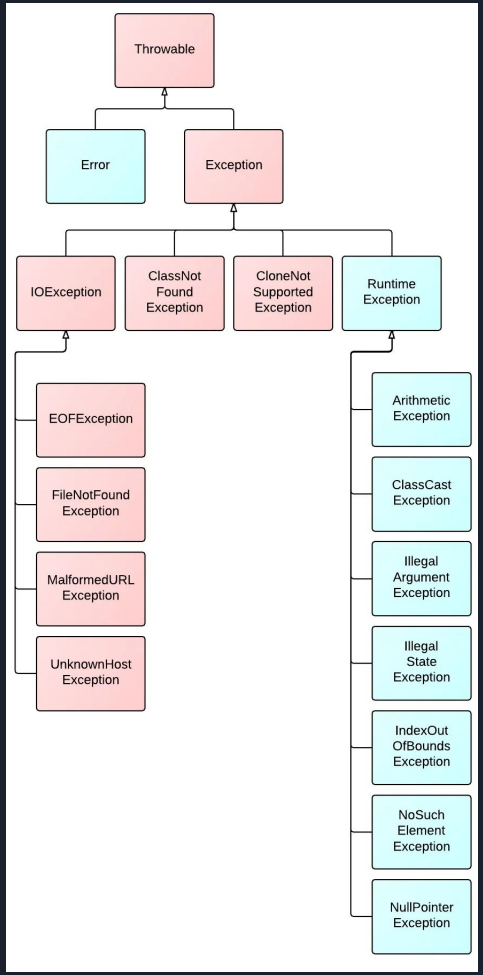
No todas las excepciones se tienen que monitorizar:

En azul las excepciones (y errores) que no se chequean y que en principio no hay por qué monitorizarlas.

En rojo las excepciones que hay que manejar, bien capturándolas con un bloque try-catch o bien relanzándolas.

Extra: 10 preguntas frecuentes con Excepciones:

ENLACE



Operaciones básicas con ficheros

- Las cinco operaciones
  - Abrir: inicio de las operaciones con el recurso.
  - Leer: obtener información del recurso.
  - Escribir: introducir información nueva en el recurso.
  - Saltar: mover el puntero de donde se lee o escribe información.
  - Cerrar: liberación del recurso.

- Tipos de acceso
  - Secuencial: se pretende que el recurso se lea de forma ordenada, empezando en principio del fichero, y escribiendo desde el principio o desde el final.
    - Nota: write/append (tanto operación como modo de apertura del escritor)
  - Aleatorio: se puede acceder a cualquier punto del fichero.

Acceso Secuencial (en Java.IO)

La API java.io ofrece varias clases para tratar con ficheros de una forma sencilla. Las principales características de estas clases son como tratan con los ficheros. Las clases se pueden separar según los siguientes criterios:

-Separaciones de uso: lectura/escritura.

-Separación de tipo de dato: bytes o texto.

-Separación de fuente de datos: ficheros, memoria y pipes.

-Separación por técnica: stream o buffer.

-Utiliza recodificación

Acceso Secuencial (en Java.IO)

Clases básicas de acceso

	Fuente de datos	Lectura	Escritura
Flujo binario	Ficheros	FileInputStream	FileOutputStream
	Memoria (byte[])	ByteArrayInputStream	ByteArrayOutputStream
	Tuberías	PipedInputStream	PipedOutputStream
Flujo de texto	Ficheros	FileReader	FileWriter
	Memoria (char[])	CharArrayReader	CharArrayWriter
	Memoria (String)	StringReader	StringWriter
	Tuberías	PipedReader	PipedWriter

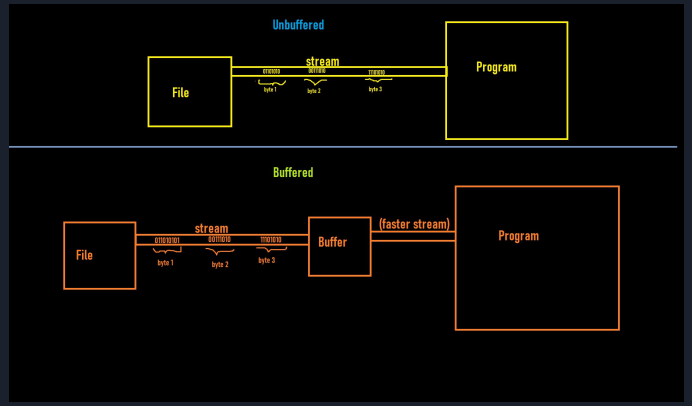
Acceso Secuencial (en Java.IO)

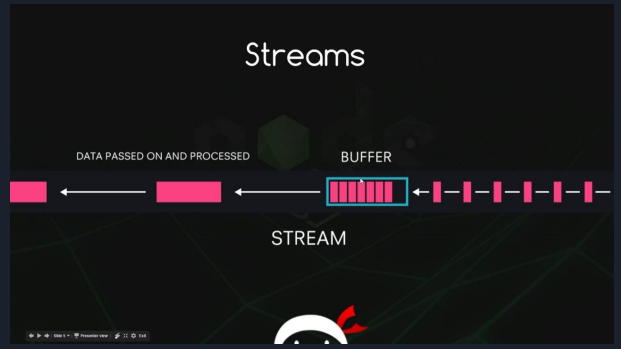
-Separación por técnica: stream o buffer.

Stream es la técnica que usa directamente el flujo de datos.

Se puede combinar con un buffer, es decir, una estructura de memoria intermedia que guarda los datos.

Esta técnica proporciona mayor eficiencia y velocidad, a cambio de una estructura intermedia extra a usar y a mantener (no ocupa mucho espacio y se puede regular).





Acceso Secuencial (en Java.IO)

Clases de buffering:

	Lectura	Escritura
Flujo binario	BufferedInputStream	BufferedOutputStream
Flujo de texto	BufferedReader	BufferedWriter

Ejemplo de uso:

Sin buffer: new FilerReader("fich.txt")

Con buffer: new BufferedReader(new FilerReader("fich.txt"))

Acceso aleatorio

Para acceder a cualquier posición de un fichero en cualquier momento, la API nos ofrece la clase RandomAccessFile (Manual Web).

Los objetos de esta clase sirven tanto para lectura como para escritura. Ej:

```
RandomAccessFile fichero = new RandomAccessFile(f, "rw");
```

Para posicionarnos en algún punto del fichero, podemos hacerlo indicando a qué byte queremos ir con la función seek(). Ej:

```
fichero.seek(5);// nos situamos en el byte 5

fichero.seek(fichero.length()); //nos situamos al final del fichero
```

Tip: la clase Properties

Para guardar muchos datos o escalar los sistemas de persistencia hay alternativas mejores a los ficheros que se adaptan mejor a las necesidades habituales.

Uno de los usos más comunes de los ficheros en las aplicaciones es el uso de un fichero de texto para guardar propiedades y configuraciones de la aplicación.

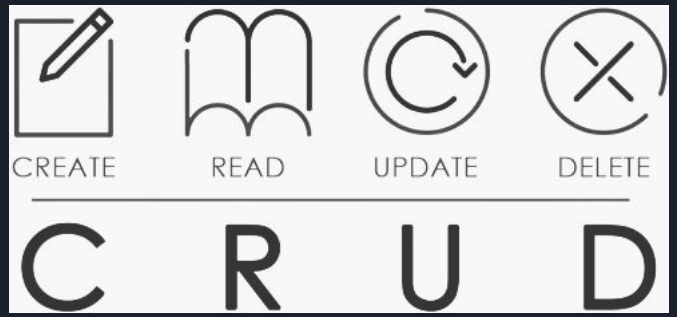
Se utiliza así ya que es un fichero sencillo de usar, que se lee pocas veces en una ejecución y de esta forma se puede configurar a mano por un técnico o un usuario avanzado.

La clase Properties (Manual) brinda al desarrollador la API para utilizar estos ficheros de texto con parejas atributo-valor de una forma sencilla y rápida.

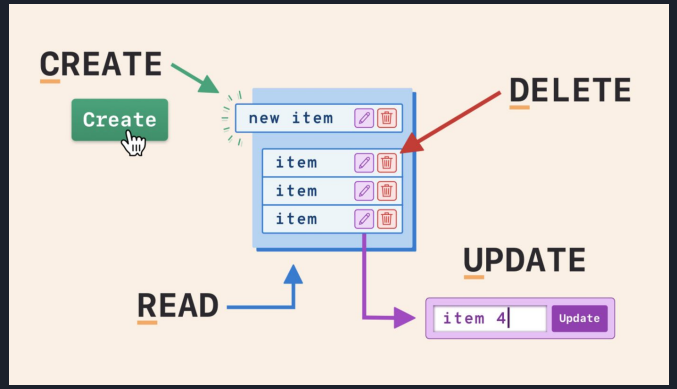
Ejemplo de uso: https://mkyong.com/java/java-properties-file-examples/

Gestión básica de datos: CRUD

CRUD corresponde con el acrónimo formado con las cuatro operaciones básicas de datos.



Cuando se trabaja con datos, es de mucha importancia pensar y diseñar cómo se van a realizar estas operaciones.



Patrón de diseño MVC

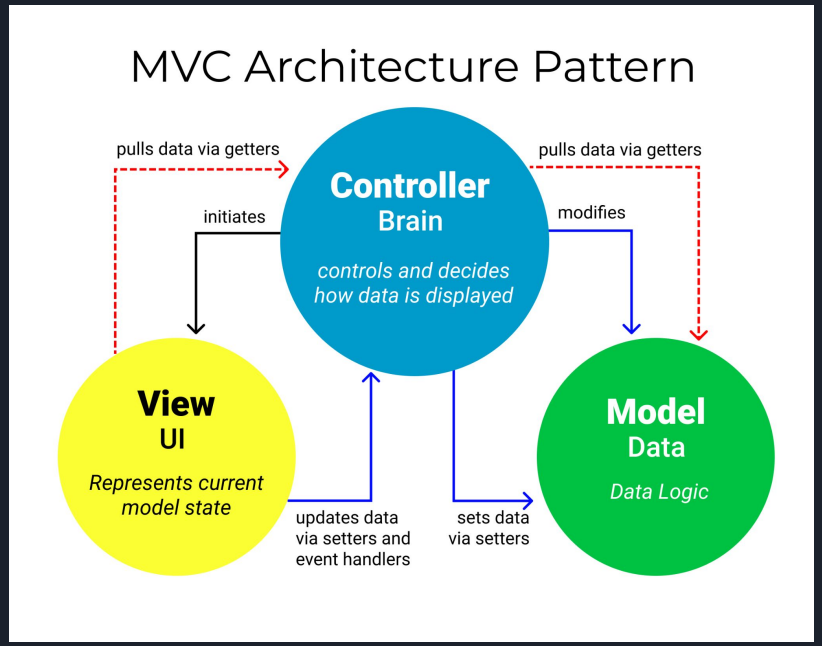
Patrón de diseño ampliamente utilizado.

En este patrón, el grueso de la aplicación se parte en tres:

-Modelo: la parte del código encargada en representar los datos y manejar su manipulación básica y persistencia.

-Vista: parte del código que representa el estado del modelo de cara al usuario y gestiona la interacción con él.

-Controlador: parte central del código que se dedica al control del sistema (inicialización, comunicación entre las partes, etc.).



Bibliografía

- Cortijo Bon, C. A, Acceso a Datos

Recursos online:

- Java 8 API
  - https://docs.oracle.com/javase/8/docs/api/
  - https://docs.oracle.com/javase/tutorial/essential/io/index.html
- Data Flair (con tutoriales de algunas clases de Java)
  - https://data-flair.training/blogs/java-file-class/
- Diagram of Exception Hierarchy
  - https://www.programcreek.com/2009/02/diagram-for-hierarchy-of-exception-classes/
- Blogs con explicación de CRUD:
  - https://www.kyocode.com/2019/11/crud-que-es/
  - https://medium.com/geekculture/crud-operations-explained-2a44096e9c88
- Blog con explicación de MVC:
  - https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/